R3 · THE CONVERSATIONAL SHOPPING ASSISTANT THAT DOESN'T LIE · TEAM DINO (1086) · 31 OCT 2026

Selectra

A product search and shopping assistant for a fixed 5,000-product catalogue and a 30-section store policy. Shoppers write short, misspelt, often Hinglish questions. The assistant must answer, ask which product they mean, or refuse, and every fact it states must come from the catalogue or the policy.

What we are scored on

Each of the 150 test questions needs one action: `answer`, `clarify` or `refuse`. The official `evaluate.py` scores every question like this:

```
question_score = action_correct × (0.4 + 0.3 × grounded + 0.3 × helpful
```

−3
for every price not copied from the catalogue

≤ 10
cited products, or product credit is lost

0.73
baseline score on the 40 dev questions

20%
of judging is this score; the rest is framing, soundness, usability, responsibility, demo

The rule everything follows

> **The LLM interprets the question. Code checks every fact against the catalogue. The catalogue decides.**

The language model never sees a price and never writes one. It turns the shopper's words into a structured search plan. Everything after that is deterministic code that reads the catalogue record, so any value we show can be traced to a field in a specific product.

How a question flows through the system

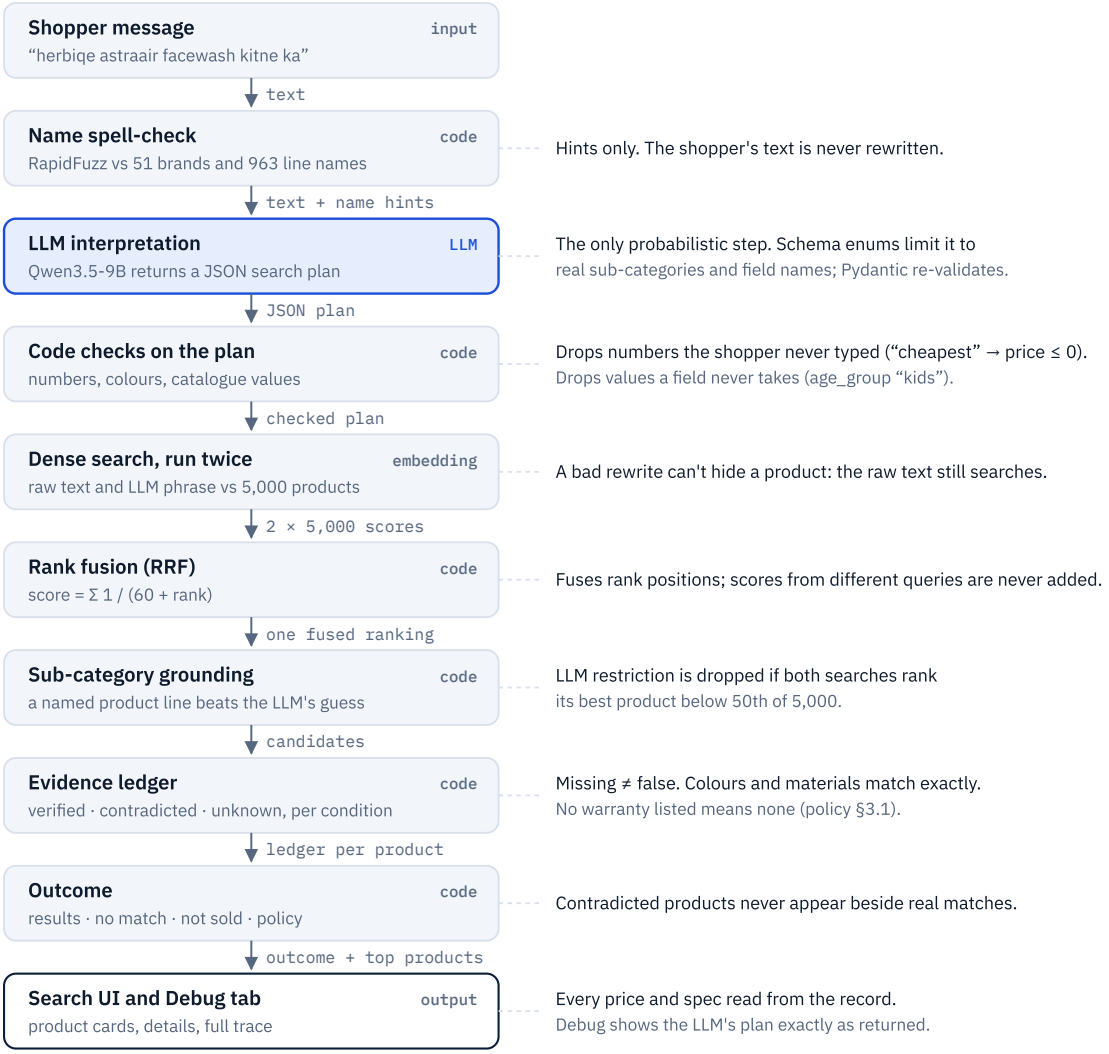


Figure 1. One LLM call per question; every later stage is code or a fixed embedding model. The right column is the guard each stage adds against stating something the catalogue does not say.

Decisions we made, and the evidence

DECISION	WHY
LLM first	Descriptions such as “baal sukhane wali machine” fail with embeddings alone: hit@1 rose from 0.51 to 0.89 when the LLM interprets first.
Search twice, then fuse	The LLM's phrase alone loses named products (dev hit@1 0.85 vs 0.93 for raw text). Fusing both keeps the best of each.
Catalogue picks the sub-category	The LLM guesses product types from brand names wrongly (“Funkari TerraFusion” called a jigsaw puzzle). Trusting it alone cut named-product hit@1 to 0.78.
No reranker	A correct product is already in the top 30 for every probe query. A reranker scores relevance and cannot tell a 50 ml from a 150 ml variant, which is a clarify case.
No vector database	5,000 × 1,024 floats is about 20 MB; a NumPy dot product over all of it takes about 1 ms.
LLM thinking off	Thinking mode used about 2,000 tokens and 28 s per question with no better JSON. Without it: about 120 output tokens.

Measured retrieval quality

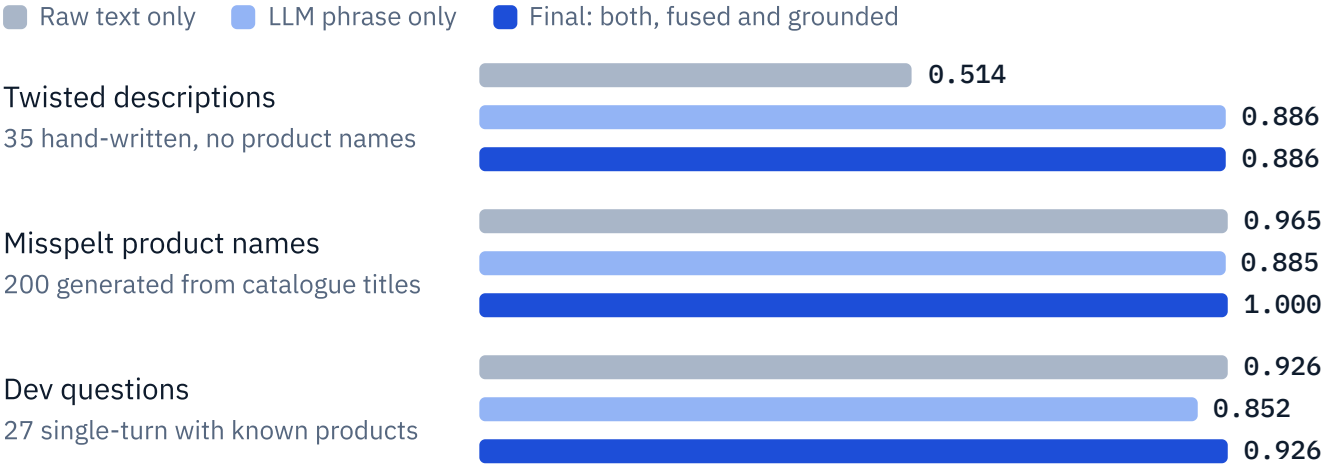


Figure 2. hit@1: share of questions whose top result is a correct product (bars to scale, 0 to 1). Raw text fails on descriptions, the LLM phrase alone fails on names, and the final pipeline is best or tied on all three sets. Also measured: precision of the top 10 on twisted questions rose from 0.49 to 0.88, and all 16 requests for things the store doesn't sell were recognised.

≈ 3,000 / 120
LLM tokens in / out per question

1.5 – 2.5 s
per question on a healthy server; the LLM call is ~90% of it

≈ 1 ms
similarity search over all 5,000 products

262
probe queries re-run after every change

How the outcome is decided

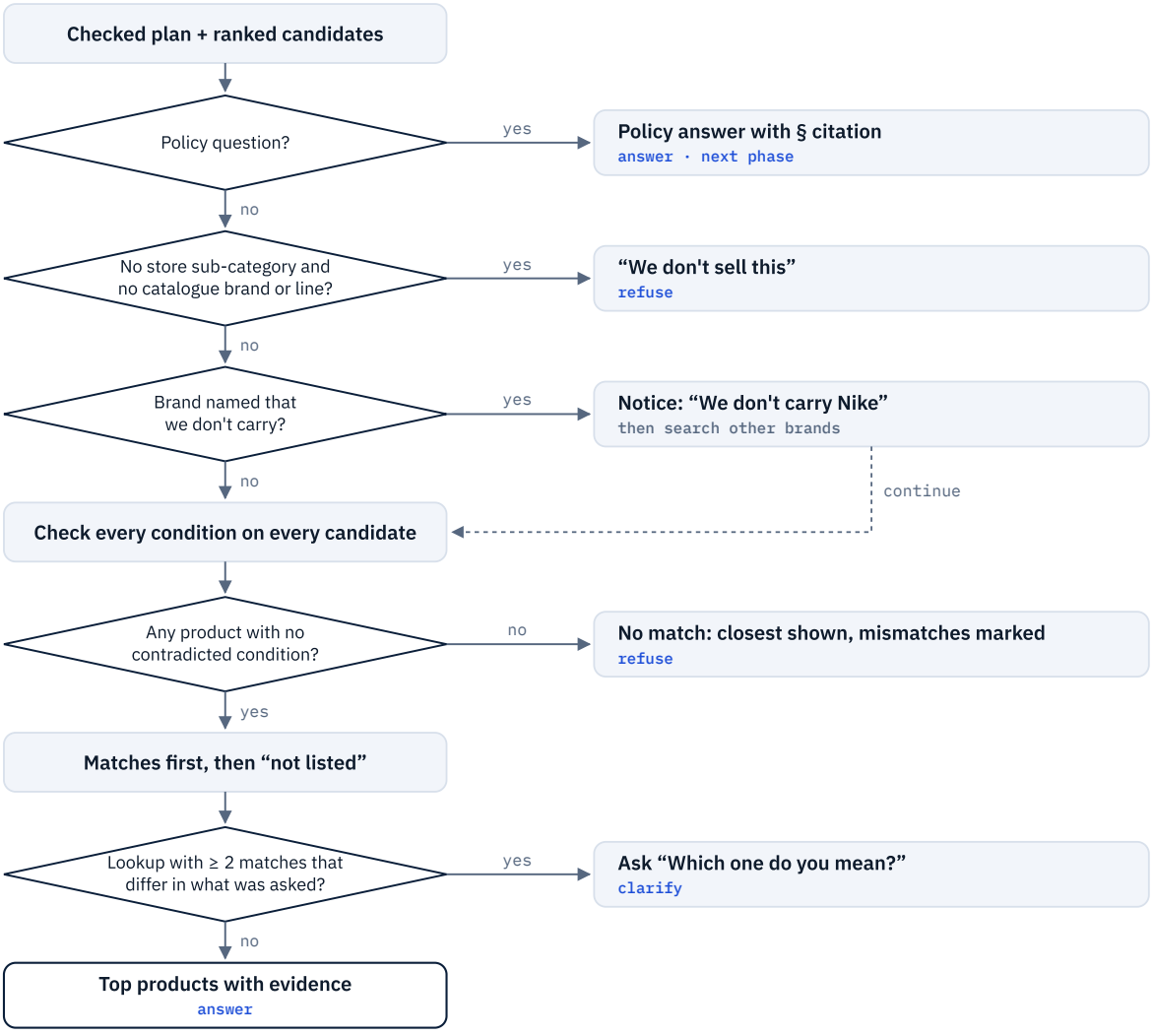


Figure 3. Every branch is a rule in code, in this order. Blue labels are the competition action each branch maps to. The demo UI already shows these states; writing them into the submission file is the next phase.

Which source decides the sub-category

THE SHOPPER NAMED	SUB-CATEGORIES SEARCHED	EXAMPLE
Brand and product line	Whatever the catalogue lists for that line. The LLM's guess is ignored.	`funkari terrafusion 500 piece` → Building Blocks
Only a line name	The line's sub-categories, plus the LLM's if they conflict	`rang` is a line name and a Hindi word
Only a description	The LLM's sub-categories, unless both searches disagree strongly	`kapde press karne wala` → Irons
Nothing the store sells	None. Refuse.	`silver payal dikhao`

Stack

- **LLM:** Qwen3.5-9B Instruct, GGUF Q4_K_M, served by Ollama on a LAN GPU server. Thinking off, 8,192-token context.
- **Embeddings:** `qwen3-embedding:0.6b`, 1,024 dimensions, normalised. Instruction prefix on queries only.
- **Index:** NumPy matrix cached as `.npy`; rebuilt automatically when the catalogue or model changes.
- **Also:** RapidFuzz for names, Pydantic for the plan schema, NiceGUI for the UI, uv with Python 3.13.

Run it

```
# web UI → http://localhost:8080
cd FULL_CODE
uv run python -m app.main

# terminal search with full trace
uv run python -m src.search_cli "chea

# retrieval quality on 262 probe quer
uv run python -m src.evaluation.retri
```

Where the code lives

```
FULL_CODE/
  src/config.py                          server URL, model names, paths (env-overr
  src/models/llm.py                      schema-constrained JSON calls, retry, tok
  src/models/embedder.py                 batched embeddings, query instruction
  src/preprocessing/products.py          catalogue loading, search text, vocabular
  src/retrieval/name_matcher.py          fuzzy brand and line hints
  src/retrieval/dense.py                 cached embedding index
  src/retrieval/product_search.py        dual search, RRF, sub-category grounding
  src/conversation/query_interpreter.py  LLM plan, prompt, code checks
  src/verification/evidence.py           Evidence Ledger
  src/search_service.py                  the pipeline the UI calls
  src/evaluation/                        retrieval probe and probe query sets
  app/                                   NiceGUI UI: main, product_view, debug_vie
```

Status

PART	STATE	NOTES
Query interpretation	DONE	LLM plan with code checks on numbers, colours and catalogue values
Product retrieval	DONE	Dual search, RRF, grounding, disagreement guard
Evidence Ledger	DONE	Conditions and asked fields, three states, source field recorded
Demo UI	DONE	Search and Debug tabs
Answer / clarify / refuse	DONE	Figure 3 as code, following README §6
Response writer	DONE	Code fills IDs, `stated_prices`, `stated_facts` and the text from templates; price guard before output
Policy answers	DONE	LLM picks a real section ID or says the policy doesn't cover it
Multi-turn	DONE	Earlier turns passed to the interpreter; budgets typed earlier still count
Submission and score	DONE	Dev score 1.000 with `evaluate.py` (baseline 0.73), 0 hallucinated prices; test file written
Material backstop	NEXT	Code check for material words the LLM drops

Known risks

- **Offline rule.** Models must run offline during judging. Confirm with the organisers that a LAN server counts.
- **Server stability.** On 30 Oct the GPU slowed from 73 to about 16 tokens/s and crashed once with a CUDA error. Check it before any demo.
- **Material words.** The LLM sometimes drops a material (“red silk saree”). Colour has a code backstop; material does not yet.
- **Small labelled set.** Only 40 dev questions have answers. Our probe sets are self-written, so their numbers may flatter us.

Prepared for team dino (1086), 31 Oct 2026. Figures come from `src/evaluation/retrieval_probe.py` and the dev set; the catalogue and policy are synthetic data from the competition pack.